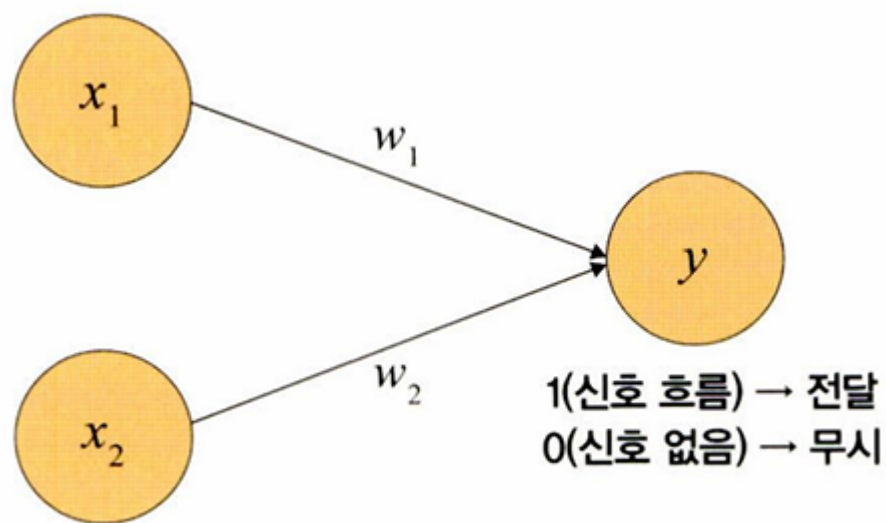


4장_딥러닝 시작

4.1. 인공 신경망의 한계와 딥러닝 출현

퍼셉트론

- 프랭크 로젠블라트가 1957년 고안한 선형 분류기
- 입력층, 출력층, 가중치로 구성된 오늘날 신경망의 기원이 되는 알고리즘



- 입력: 흐름이 있는 다수의 신호 → 출력: 하나의 신호
- 출력 신호의 의미
 - '1' → 정보를 앞으로 전달
 - '0' → 정보를 전달하지 않음

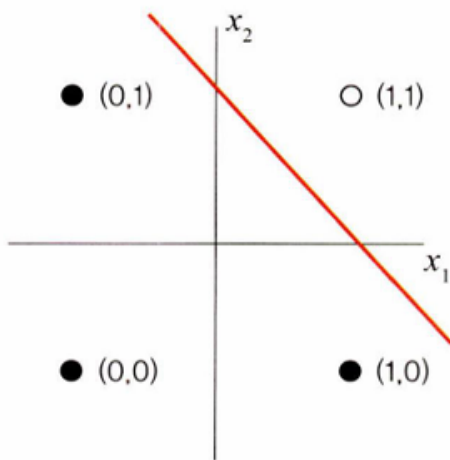
컴퓨터가 입력을 논리적으로 인식하는 방식 - 논리 게이트

- AND 게이트
- OR 게이트

▼ 표 4-1 AND 게이트

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

▼ 그림 4-2 AND 게이트

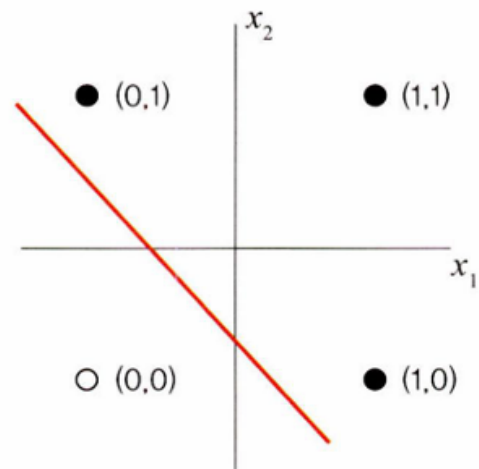


AND 게이트를 데이터 분류로 표현한 것

▼ 표 4-2 OR 게이트

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1

▼ 그림 4-3 OR 게이트



- XOR 게이트 - 배타적 논리합

▼ 표 4-3 XOR 게이트

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

: 입력 두 개 중 한 개만 '1'일 때 작동하는 논리 연산

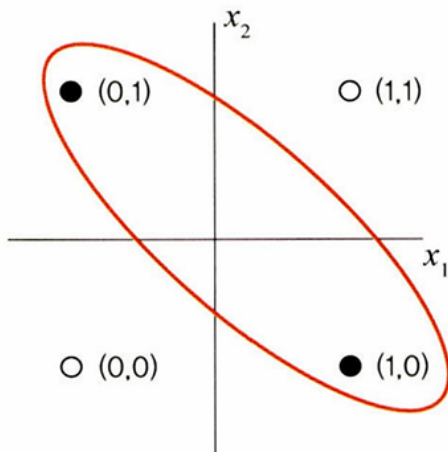
→ 데이터가 비선형적으로 분리되므로 단층 퍼셉트론만으로는 제대로 된 분류가 어려움

⇒ 다층 퍼셉트론 (multi-layer perceptron)

입력층과 출력층 사이에 하나 이상의 중간층(은닉층)을 두어 비선형적으로 분리되는 데이터에 대해서도 학습이 가능하도록 함

- 심층 신경망(DNN): 은닉층이 여러 개 있는 신경망 == 딥러닝!

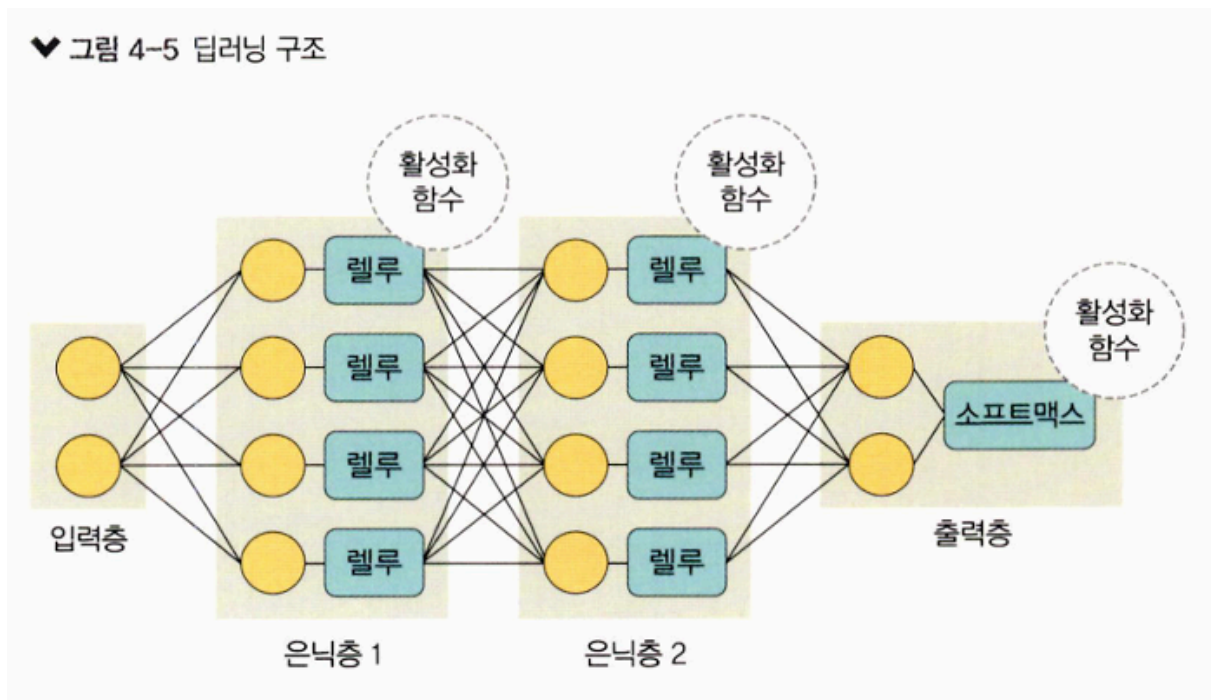
▼ 그림 4-4 XOR 게이트



4.2. 딥러닝 구조

딥러닝: 여러 층을 가진 인공 신경망을 사용하여 학습을 수행하는 것

▼ 그림 4-5 딥러닝 구조



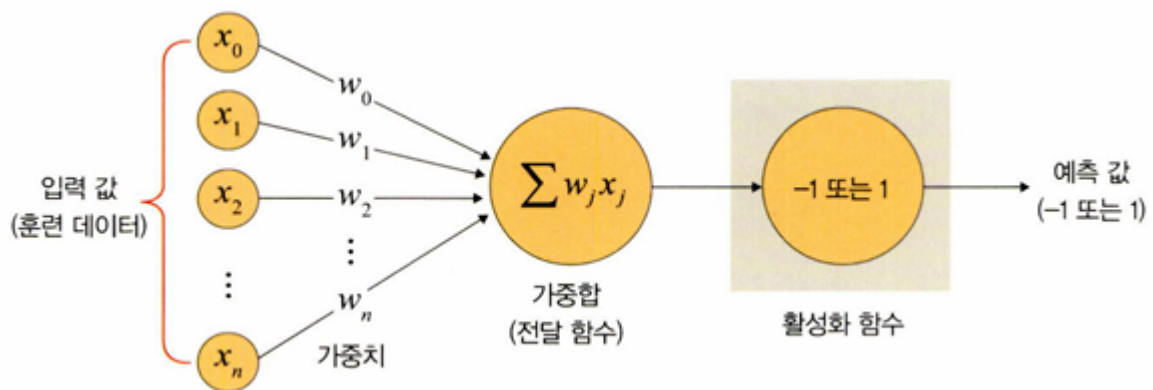
4.2.1. 딥러닝 용어

구분	구성 요소	설명
층	입력층(input layer)	데이터를 받아들이는 층

	은닉층(hidden layer)	모든 입력 노드부터 입력 값을 받아 가중합 을 계산하고 이 값을 활성화 함수에 적용 하여 출력층에 전달 하는 층
	출력층(output layer)	신경망의 최종 결과값이 포함된 층
가중치 (weight)		노드와 노드 간 연결 강도
편향 (bias)		가중합에 더해 주는 상수. 최종적으로 출력되는 값을 조절하는 역할
가중합 (weighted sum), 전달 함수		(가중치) X (신호)
함수	활성화 함수 (activation function)	신호를 입력받아 이를 적절히 처리해 출력해 주는 함수
	손실 함수 (loss function)	가중치 학습을 위한 출력 함수의 결과와 실제 값 간의 오차 측정 함수

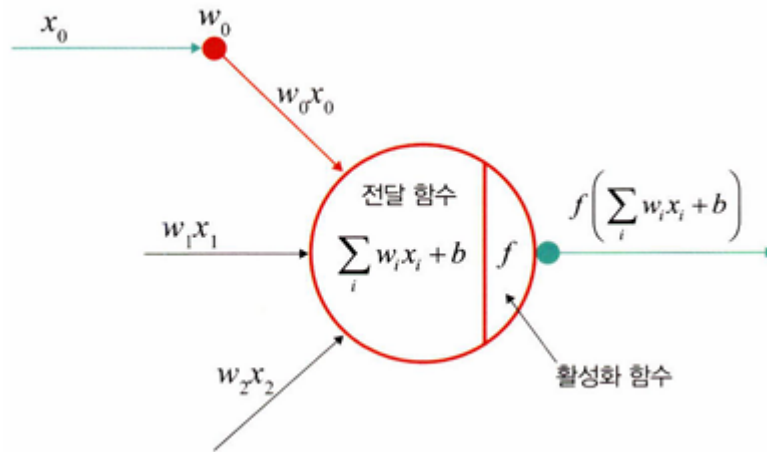
가중치

: 입력 값이 연산 결과에 미치는 영향력 조절하는 요소



가중합, 전달 함수(transfer function)

: 가중합을 계산해서 활성화 함수로 보냄



활성화 함수

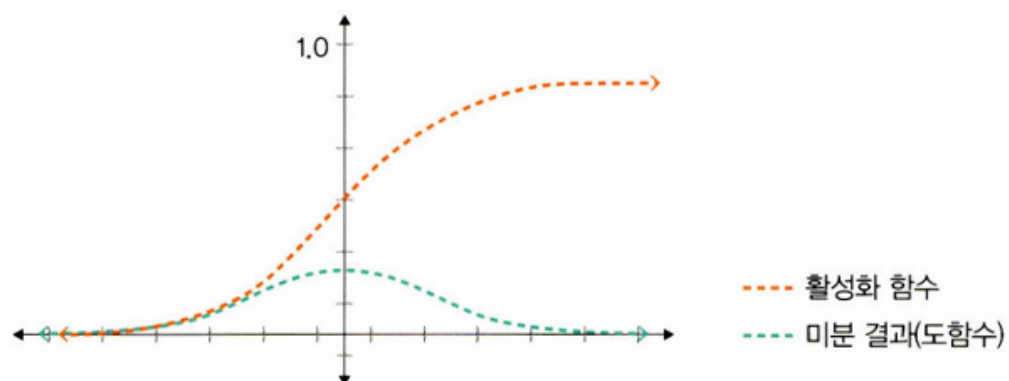
: 전달 함수에서 전달받은 값을 출력할 때 일정 기준에 따라 출력 값을 변화시키는 비선형 함수

— 비선형 함수: 직선으로 표현할 수 없는 데이터 사이의 관계를 표현하는 함수

- 시그모이드 함수 → 0~1 사이의 비선형 형태로 변형

$$f(x) = \frac{1}{1 + e^{-x}}$$

♥ 그림 4-8 시그모이드 활성화 함수와 미분 결과



- 로지스틱 회귀 등 분류 문제를 확률적으로 표현하는 데 사용됨

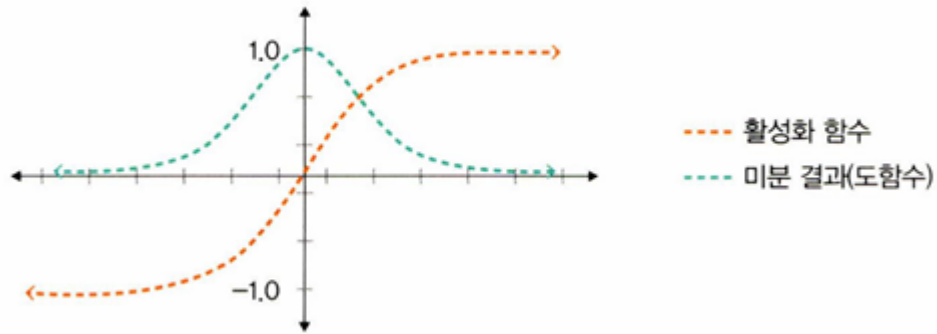
▼ ⚠ 기울기 소멸 문제(vanishing gradient problem): 딥러닝 모델의 깊이가 깊어지면 기울기가 사라지는 문제 발생 → 딥러닝 모델에서는 잘 사용하지 X

- 시그모이드의 도함수 최댓값 = 0.25

- 역전파 과정에서 그래디언트가 출력층 → 입력층으로 전파됨, 각 층의 가중치와 활성화 함수의 도함수를 곱해 전달되므로 0.25 이하의 값이 계속 곱해지면서 매우 작아짐

- **하이퍼볼릭 탄젠트 함수** → -1~1 사이의 비선형 형태로 변형

▼ 그림 4-9 하이퍼볼릭 탄젠트 활성화 함수와 미분 결과

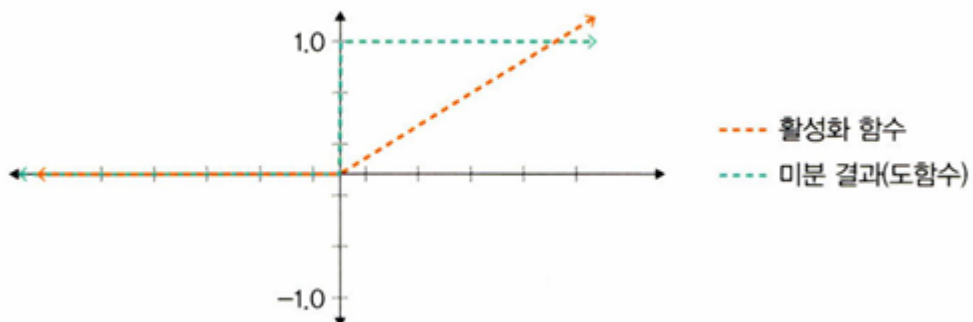


- 결괏값 평균이 양수로 편향되지 않음

⚠ 기울기 소멸 문제

- **렐루(ReLU) 함수** → 입력(x)이 음수면 0 출력, 양수면 x 출력

▼ 그림 4-10 렐루 활성화 함수와 미분 결과



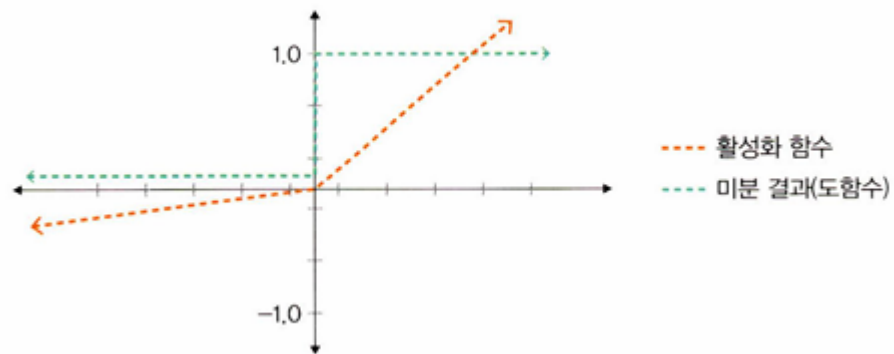
- 경사하강법에 영향을 주지 않음 → 학습 속도가 빠르고(하이퍼볼릭 탄젠트 함수의 6배), 기울기 소멸 문제가 발생하지 않음

- 일반적으로 은닉층에서 사용

⚠ 음수 값을 입력받으면 항상 0 출력 → 학습 능력 감소 ⇒ 리키 렐루 함수 등으로 해결

- **리키 렐루(Leaky ReLU) 함수** → 입력 값이 음수면 0.001 등 매우 작은 수를 반환

▼ 그림 4-11 리키 렐루 활성화 함수와 미분 결과



- 입력 값 수렴 구간 제거 → 렐루 함수의 문제 해결
- **소프트맥스(softmax) 함수** → 입력 값을 0~1 사이에 출력되도록 정규화하여 출력 값들의 총합이 항상 1이 되도록 함
 - 수식

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)}$$

- $\exp(x)$: 지수 함수(exponential function)
- n : 출력층 뉴런 개수
- y_k : k번째 출력

⇒ 분자: 입력 신호 a_k 의 지수 함수 / 분모: 모든 입력 신호의 지수 함수 합

```
class Net(torch.nn.Module):
    def __init__(self, n_feature, n_hidden, n_output):
        super(Net, self).__init__()
        self.hidden = torch.nn.Linear(n_feature, n_hidden) ----- 은닉층
        self.relu = torch.nn.ReLU(inplace=True)
        self.out = torch.nn.Linear(n_hidden, n_output) ----- 출력층
        self.softmax = torch.nn.Softmax(dim=n_output)
    def forward(self, x):
        x = self.hidden(x)
        x = self.relu(x) ----- 은닉층을 위한 렐루 활성화 함수
```

```
x = self.out(x)
x = self.softmax(x) ----- 출력층을 위한 소프트맥스 활성화 함수
return x
```

렐루 함수, 소프트맥스 함수 구현 코드

손실 함수

: 학습을 통해 얻은 데이터의 추정치가 실제 데이터와 얼마나 차이가 나는지 평가하는 지표

- **평균 제곱 오차(Mean Squared Error, MSE)**

: 실제 값과 예측 값의 차이(error)를 제곱하여 평균 낸 것

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\left(\begin{array}{l} \hat{y}_i: \text{신경망의 출력(신경망이 추정한 값)} \\ y_i: \text{정답 레이블} \\ i: \text{데이터의 차원 개수} \end{array} \right)$$

- 회귀에서 주로 사용
- 파이토치 MSE

```
import torch
```

```
loss_fn = torch.nn.MSELoss(reduction='sum')
y_pred = model(x)
loss = loss_fn(y_pred, y)
```

• 크로스 엔트로피 오차(CEE)

- 분류 문제에서 원-핫 인코딩 했을 때만 사용하는 오차 계산법
- 시그모이드 함수에 평균 제곱 오차를 적용하면 자연 상수 e 때문에 울퉁불퉁한 그래프가 출력됨
→ CEE 사용
- CEE 적용 → 경사 하강법 과정에서 학습이 local 최소점에서 멈출 수 있음 → 방지
차원에서 자연 로그를 모델 출력 값에 취함

$$CrossEntropy = - \sum_{i=1}^n y_i \log \hat{y}_i$$

$$\left(\begin{array}{l} \hat{y}_i: \text{신경망의 출력(신경망이 추정한 값)} \\ y_i: \text{정답 레이블} \\ i: \text{데이터의 차원 개수} \end{array} \right)$$

- 파이토치

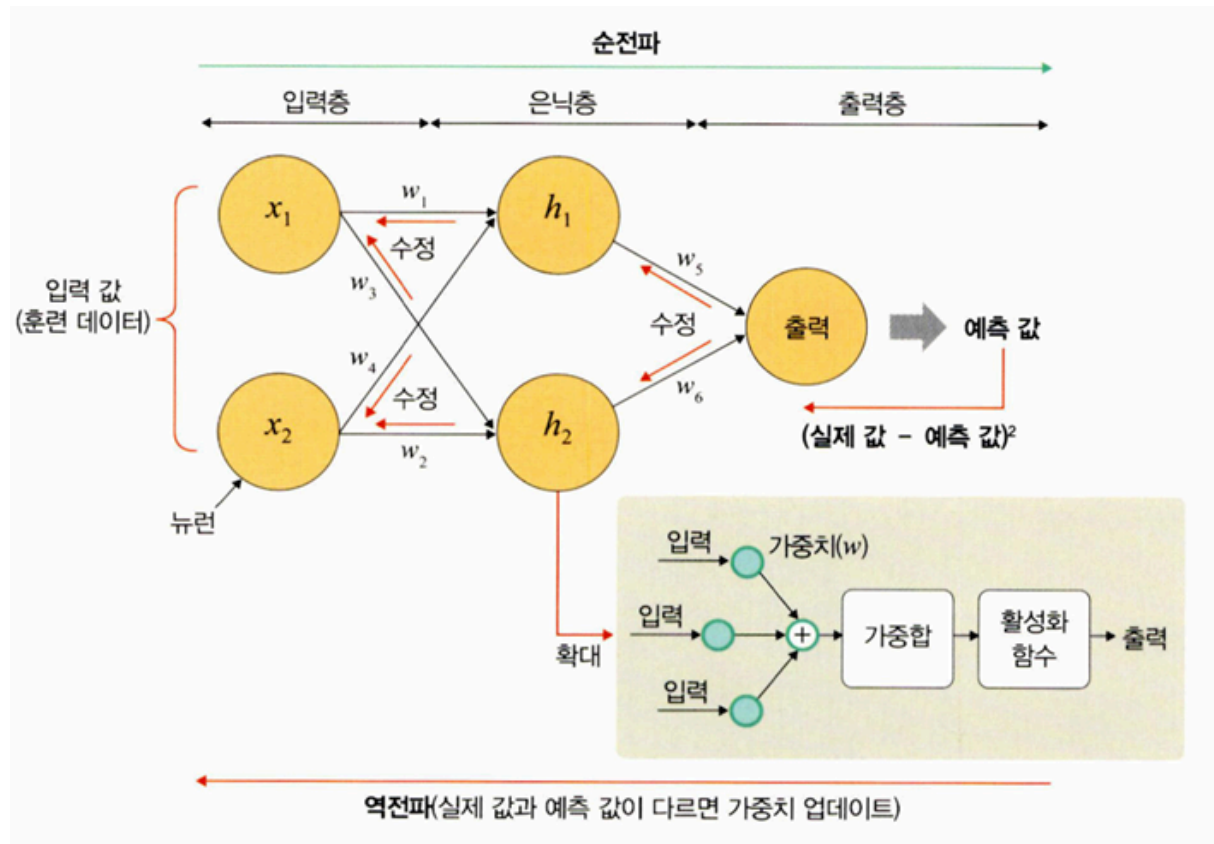
```
loss = nn.CrossEntropyLoss()
input = torch.randn(5, 6, requires_grad=True)
target = torch.empty(3, dtype=torch.long).random_(5)
output = loss(input, target)
output.backward()
```

----- torch.randn은 평균이 0이고 표준편차가 1인
가우시안 정규분포를 이용하여 숫자를 생성

----- torch.empty는 dtype torch.float32의
랜덤한 값으로 채워진 텐서를 반환

4.2.2. 딥러닝 학습

= 순전파 + 역전파



• 순전파(feedforward)

- 훈련 데이터가 들어올 때 발생
- 데이터 기반 **예측 값 계산**하기 위해 전체 신경망을 교차해 지나가는 과정
- 모든 뉴런이 이전 층의 뉴런에서 수신한 정보에 변환(가중합 및 활성화 함수)을 적용해 다음 층(은닉층)의 뉴런으로 전송하는 방식
- 계산된 예측 값은 출력층에 도달

• 역전파(backpropagation)

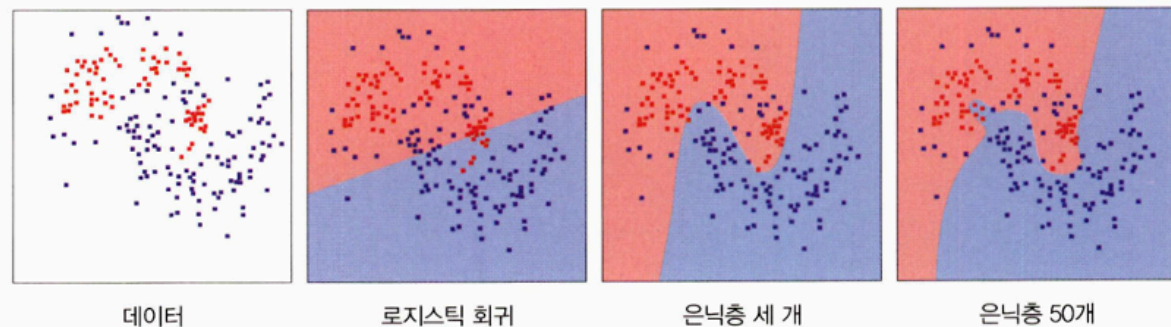
- 예측값 계산 끝나면 손실 함수가 예측 값과 실제 값의 차이를 추정
- 손실 함수 비용이 0인 것이 이상적이므로, 0에 가깝게 만들기 위해 모델이 훈련을 반복하면서 가중치 조정
- **계산된 손실 정보: 출력층 → 은닉층 → 입력층** 역으로 전파됨
 - 은닉층의 모든 뉴런으로 전파되지만, 각 뉴런의 가중치에 따라 값이 달라짐 (어떤 뉴런이 손실에 영향을 크게 줬는지를 역으로 조사하는 것)

- 예측 값과 실제 값 차이를 각 뉴런의 가중치로 미분(변화율 구하려고)한 후 기존 가중치 값에서 뺌
- 이 과정을 모든 뉴런에 대해 진행 → 계산된 각 뉴런 결과를 다시 순전파 가중치 값으로 사용

4.2.3. 딥러닝의 문제점과 해결 방안

딥러닝의 핵심은 활성화 함수가 적용된 여러 은닉층을 결합해 비선형 영역을 표현하는 것

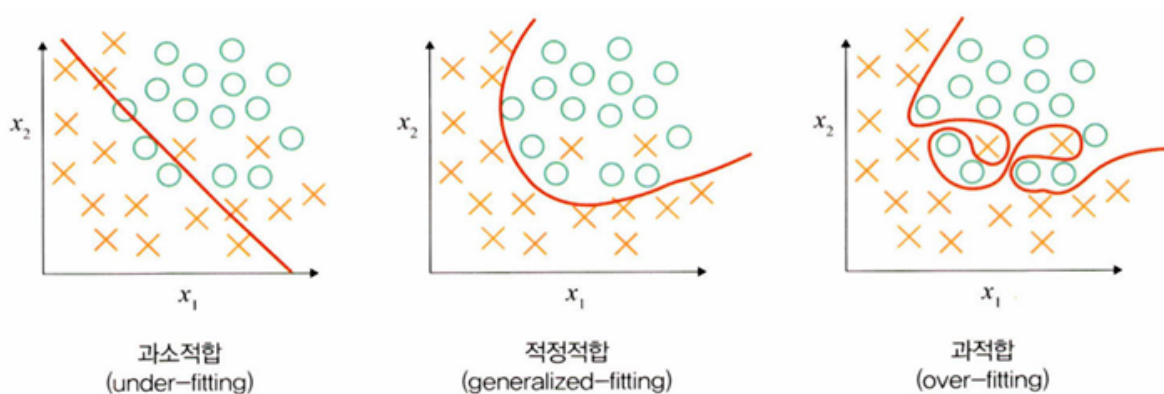
▼ 그림 4-13 은닉층이 분류에 미치는 영향



→ 활성화 함수가 적용된 은닉층 개수가 많을수록 데이터 분류가 잘 되는 것 같지만, 너무 많으면 **문제점**이 생김

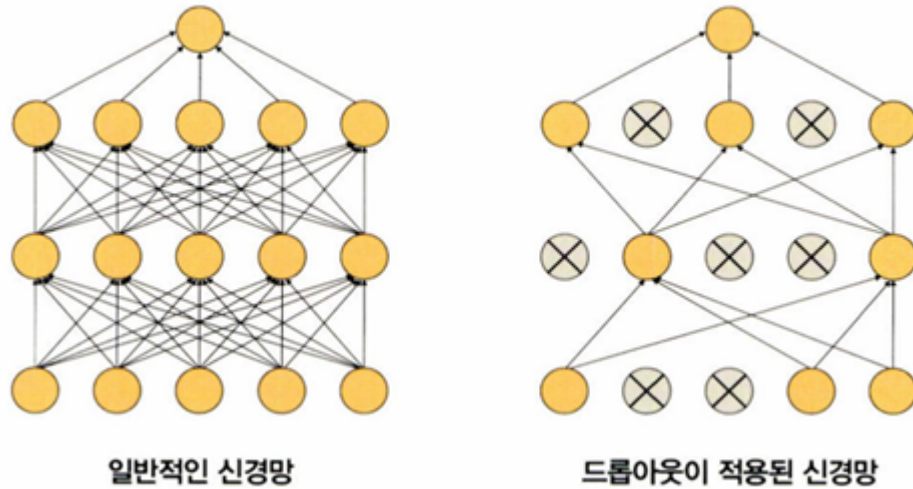
과적합(over-fitting)

훈련 데이터를 과하게 학습해 실제 데이터에 대한 오차가 증가하는 현상



- 해결책: **드롭아웃(dropout)**
 - 학습 과정 중 임의로 일부 노드들을 학습에서 제외시키는 방법

▼ 그림 4-15 일반적인 신경망과 드롭아웃이 적용된 신경망



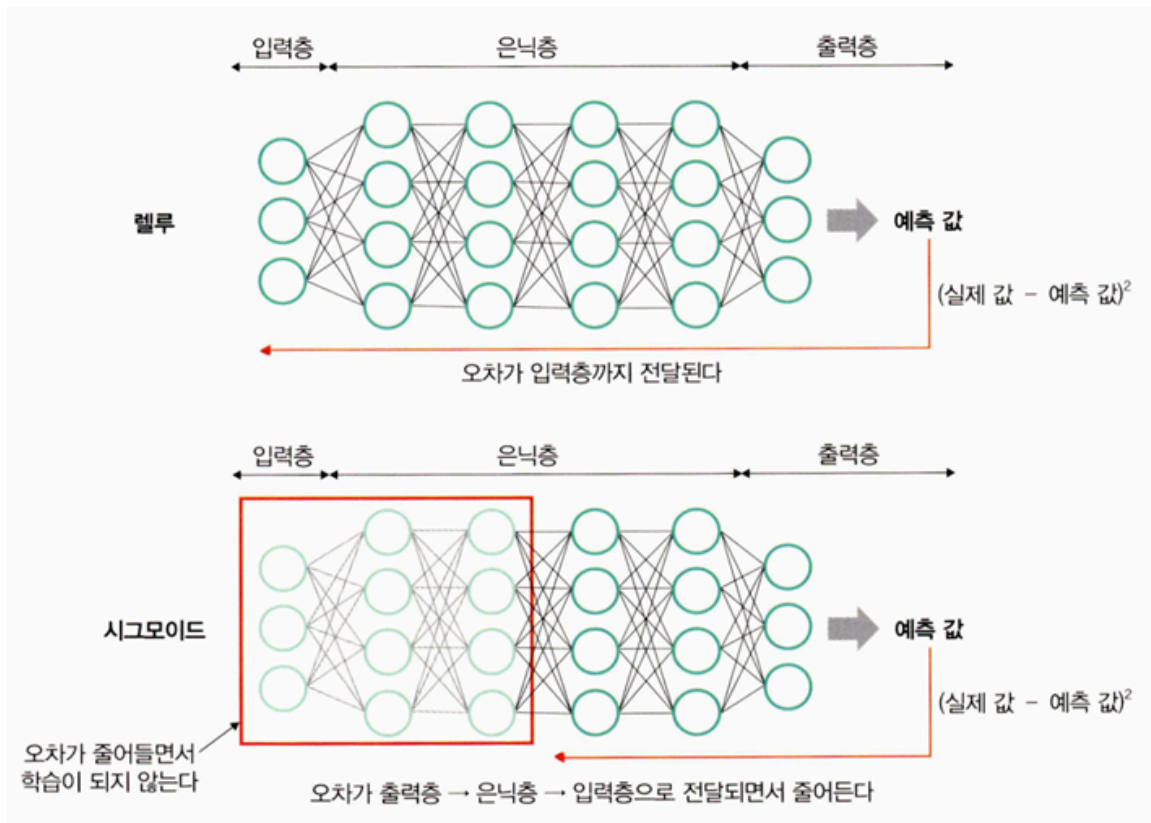
- 파이토치 구현

```
class DropoutModel(torch.nn.Module):
    def __init__(self):
        super(DropoutModel, self).__init__()
        self.layer1 = torch.nn.Linear(784, 1200)
        self.dropout1 = torch.nn.Dropout(0.5) ----- 50%의 노드를 무작위로 선택하여
        self.layer2 = torch.nn.Linear(1200, 1200)      사용하지 않겠다는 의미
        self.dropout2 = torch.nn.Dropout(0.5)
        self.layer3 = torch.nn.Linear(1200, 10)

    def forward(self, x):
        x = F.relu(self.layer1(x))
        x = self.dropout1(x)
        x = F.relu(self.layer2(x))
        x = self.dropout2(x)
        return self.layer3(x)
```

기울기 소멸 문제

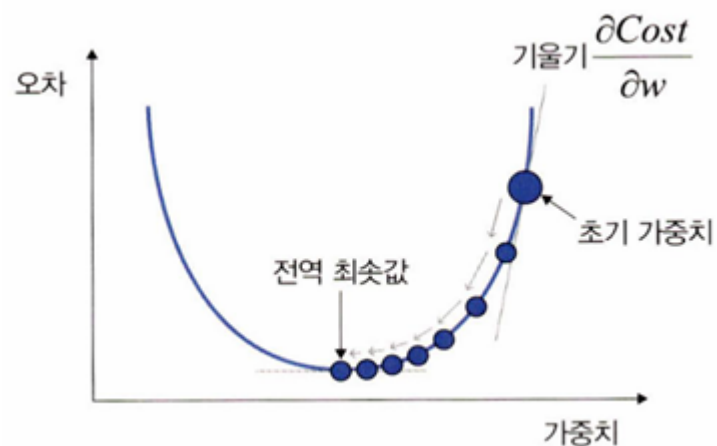
출력층에서 은닉층으로 전달되는 오차가 크게 줄어들어 학습이 되지 않는 현상



- 기울기가 소멸되기 때문에 학습되는 양이 '0'에 가까워져 학습이 더디게 진행된다. 오차를 더 줄이지 못하고 그 상태로 수렴함
- 해결책: 시그모이드, 하이퍼볼릭 탄젠트 대신 **ReLU 활성화 함수** 사용

성능 저하

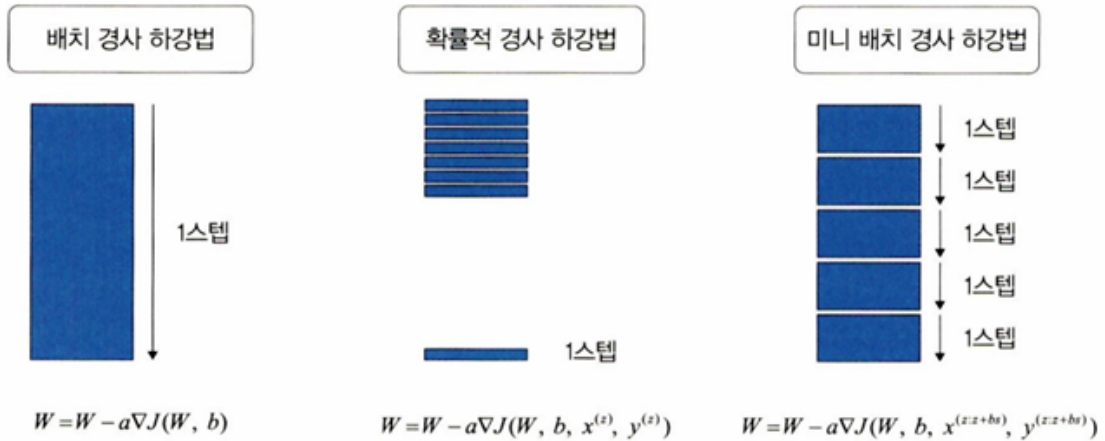
경사 하강법은 손실 함수의 비용이 최소가 되는 지점을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동시키는 과정을 반복하고, 이때 성능 저하 문제 발생



- 해결책: 확률적 경사 하강법, 미니 배치 경사 하강법

▼ 경사 하강법

▼ 그림 4-18 경사 하강법의 유형



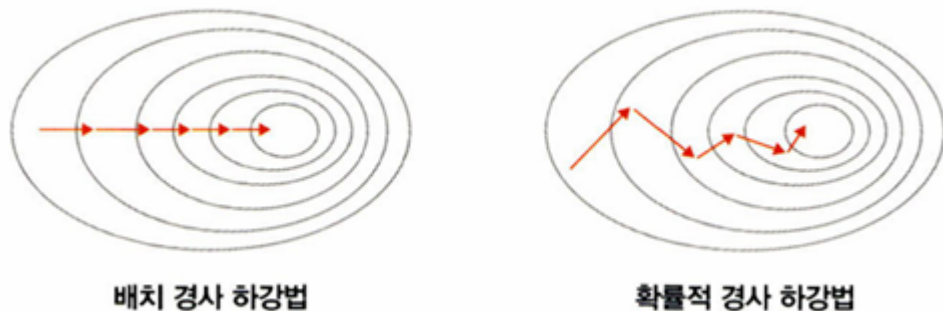
- 배치 경사 하강법(Batch Gradient Descent, BGD)
: 전체 훈련 데이터셋에 대해 가중치를 편미분하는 방법

손실 함수의 값을 최소화하기 위해 기울기(∇) 이용

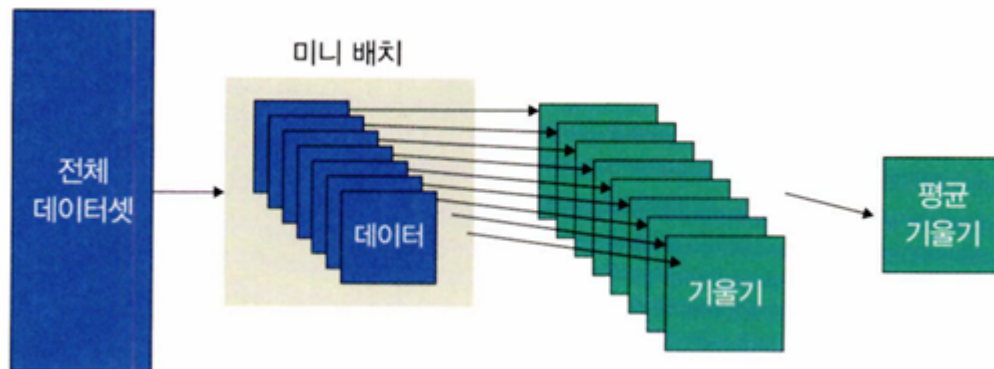
$$W = W - a \nabla J(W, b)$$

(a : 학습률, J : 손실 함수)

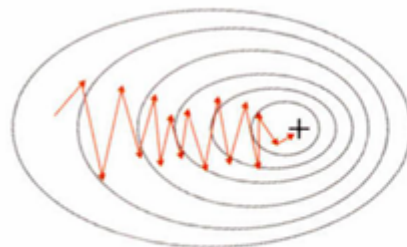
- 한 스텝에 대해 모든 훈련 데이터셋 사용 → 학습이 오래 걸림
- 확률적 경사 하강법(Stochastic Gradient Descent, SGD)
: 임의로 선택한 데이터에 대해 기울기를 계산하는 방법



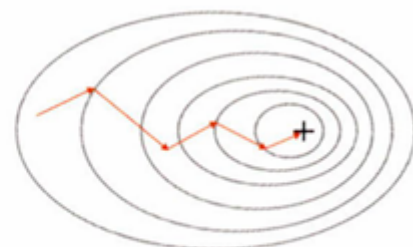
- 적은 데이터를 사용하므로 속도 빠름
- 파라미터 변경 폭 불안정
- 때로는 배치 경사 하강법보다 정확도 낮음
- 미니 배치 경사 하강법(mini-batch gradient descent)
: 전체 데이터셋을 미니 배치 여러 개로 나눔, 미니 배치들의 평균 기울기를 이용해 모델을 업데이트해서 학습하는 방법



- 배치 경사 하강법보다 속도가 빠름
- 확률적 경사 하강법보다 안정적



확률적 경사 하강법



미니 배치 경사 하강법

- 파이토치 구현

```

class CustomDataset(Dataset):
    def __init__(self):
        self.x_data = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
        self.y_data = [[12], [18], [11]]
    def __len__(self):
        return len(self.x_data)
    def __getitem__(self, idx):
        x = torch.FloatTensor(self.x_data[idx])
        y = torch.FloatTensor(self.y_data[idx])
        return x, y
dataset = CustomDataset()
dataloader = DataLoader(

```

```

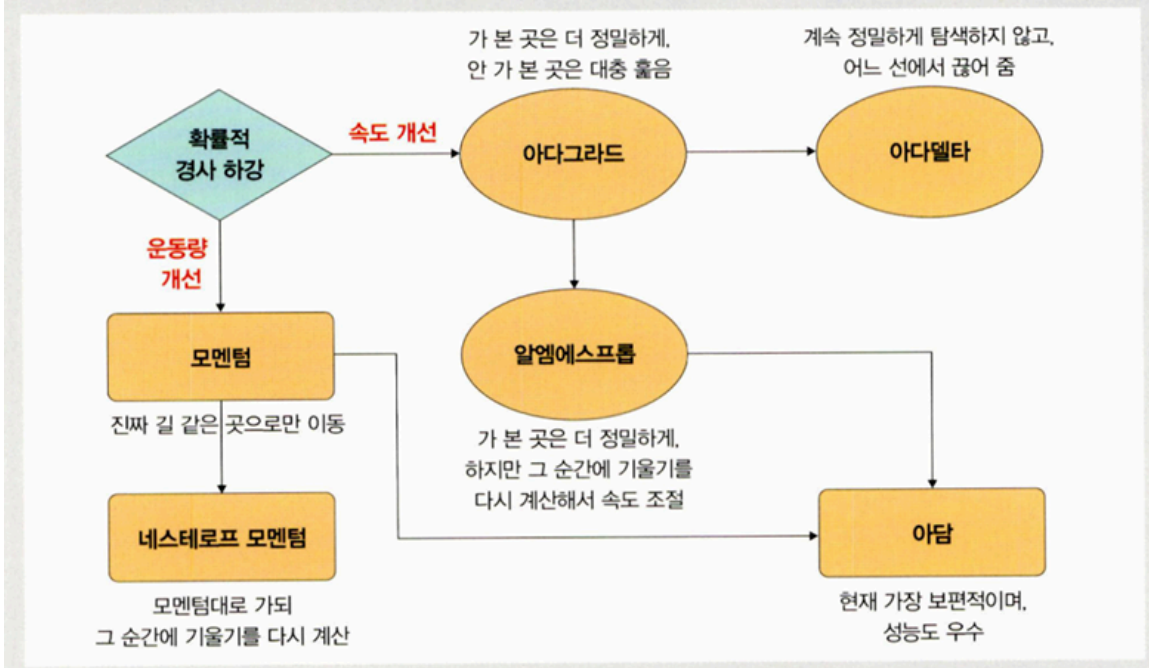
dataset, ----- 데이터셋
batch_size=2, ----- 미니 배치 크기로 2의 제공수를 사용하겠다는 의미입니다.
shuffle=True, ----- 데이터를 불러올 때마다 랜덤으로 섞어서 가져옵니다.
)

```

+) 옵티마이저(optimizer)

확률적 경사 하강법의 파라미터 변경 폭이 불안정한 문제를 해결하기 위해 **학습 속도와 운동량**을 조정하는 것

▼ 그림 4-22 옵티마이저 유형



1. 학습 속도를 조정

- **아다그라드(Adagrad, Adaptive gradient)**: 변수(가중치)의 업데이트 횟수에 따라 학습률을 조정
 - 많이 변화하지 않는 변수들의 학습률 , 많이 변화하는 변수들의 학습률

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} \nabla E(w(i))$$

$$G(i) = G(i-1) + (\nabla E(w(i)))^2$$

- G 함수: 이전 G 값(기울기 크기)의 누적
- 변수가 많이 업데이트됨 → 해당 파라미터에 대해 기울기 커짐 → G 값 커짐 ⇒ 학습률 계수 작아짐
- `optimizer = torch.optim.Adagrad(model.parameters(), lr=0.01)`
- ⚠ 기울기가 0에 수렴하는 문제 - G 값이 커짐에 따라 학습이 중단됨
- **아다델타(Adadelta, Adaptive delta)**: 아다그라드의 학습률 → D 함수(가중치의 변화량 크기를 누적인 값)로 변환

$$w(i+1) = w(i) - \frac{\sqrt{D(i-1)+\varepsilon}}{\sqrt{G(i)+\varepsilon}} \nabla E(w(i))$$

$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

$$D(i) = \gamma D(i-1) + (1-\gamma)(\Delta(w(i)))^2$$

- 아다그라드의 학습 중단 문제 해결
- `optimizer = torch.optim.Adadelta(model.parameters(), lr=1.0)`
- **알엠에스프롭(RMSProp):** 아다그라드의 G 함수 + γ 추가

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i)+\varepsilon}} \nabla E(w(i))$$

$$G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$$

- 사용자가 γ 값을 조정
 - `optimizer = torch.optim.RMSprop(model.parameters(), lr=0.01)`
2. 운동량을 조정
- **모멘텀(Momentum):** 가중치 업데이트 전 이전 수정 방향(+, -) 참고하여 같은 방향으로 일정한 비율만 수정하는 방법
 - 지그재그 현상 (+, -, +, -, ...) 감소
 - 관성 효과
 - 확률적 경사 하강법과 함께 사용

$$w(i+1) = w(i) - v(i)$$

$$v(i) = \gamma v(i-1) + \eta \nabla E(w(i))$$

기울기 → 속도

- 다음 가중치를 구할 때 이전 속도와 현재 기울기 반영

⚠ 단점: 멈추어야 할 시점에서도 관성에 의해 훨씬 멀리 이동해버릴 수 있음

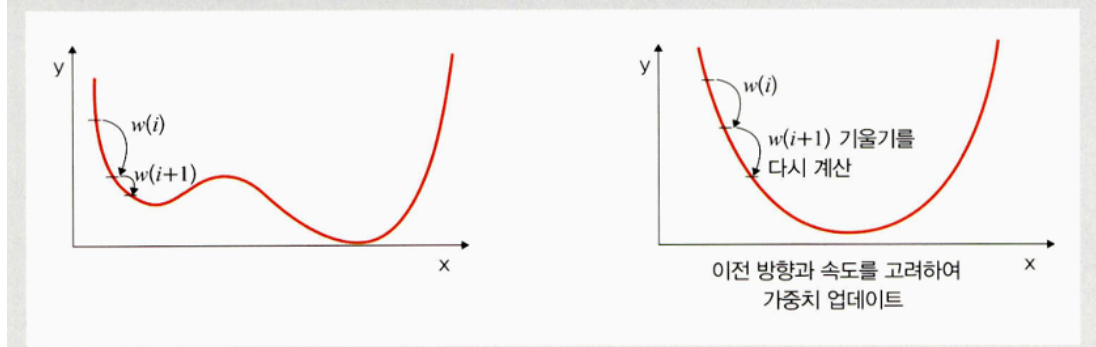
- `optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)`
- **네스테로프 모멘텀(Nesterov Accelerated Gradient, NAG):** 모멘텀 값이 적용된 지점에서만 기울기 값 계산

$$w(i+1) = w(i) - v(i)$$

$$v(i) = \gamma v(i-1) + \eta \nabla E(w(i) - \gamma v(i-1))$$

- 빠른 이동 속도 + 멈춰야 할 시점에 제동을 거는 데 용이함
- 이전 학습 속도와 현재 기울기에서 이전 속도를 뺀 변화량을 더해 가중치 계산

▼ 그림 4-23 모멘텀과 네스테로프 모멘텀



- `optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9, nesterov=True)`

3. 학습 속도 + 운동량 조정

- 아담(Adam, Adaptive Moment Estimation): 알엠에스트림의 G 함수와 모멘텀의 v(i) 사용해 가중치 업데이트

$$w(i+1) = w(i) - \frac{\eta}{\sqrt{G(i) + \epsilon}} v(i)$$

$$G(i) = \gamma_2 G(i-1) + (1 - \gamma_2)(\nabla E(w(i)))^2$$

$$v(i) = \gamma_1 v(i-1) + \eta \nabla E(w(i))$$

- `optimizer = torch.optim.Adam(model.parameters(), lr=0.01)`

4.2.4. 딥러닝 이점

특성 추출

- 머신 러닝 알고리즘: 특성 추출이 매우 복잡한 경우가 있고, 수집된 데이터에 대한 전문 지식이 필요함
- 딥러닝: 은닉층을 깊게 쌓아 파라미터를 늘린 모델 구조 → 특성 추출 과정을 알고리즘에 통합

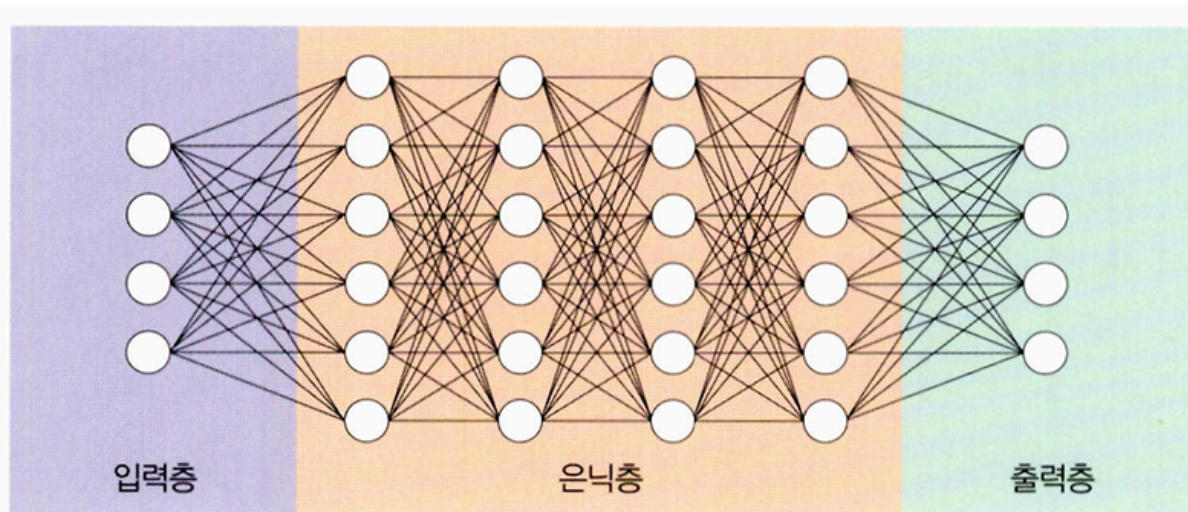
빅데이터 효율적 활용

- 딥러닝 학습을 이용한 특성 추출은 데이터 사례가 많을수록 성능 향상됨
→ 확보된 데이터가 적은 경우 머신 러닝 고려

4.3. 딥러닝 알고리즘

4.3.1. 심층 신경망(DNN)

입력층과 출력층 사이에 다수의 은닉층을 포함하는 인공 신경망



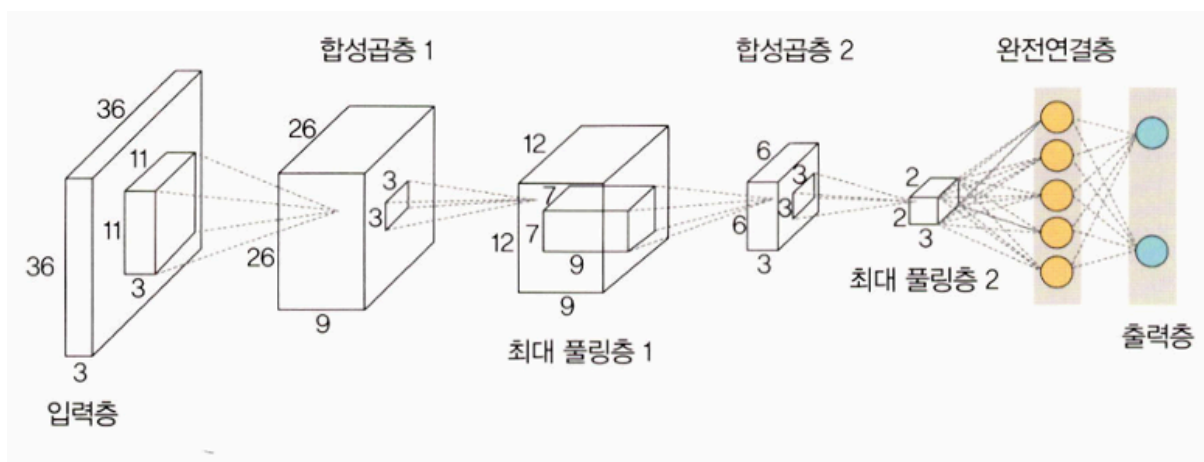
- 다수의 은닉층 → 다양한 비선형적 관계를 학습할 수 있음

⚠ 학습을 위한 연산량이 많음, 기울기 소멸 문제 등

→ 해결: 드롭아웃, 렐루, 배치 정규화 등

4.3.2. 합성곱 신경망(Convolutional Neural Network, CNN)

합성곱층(convolutional layer)과 풀링층(pooling layer)을 포함하는 이미지 처리 성능이 좋은 인공 신경망 알고리즘



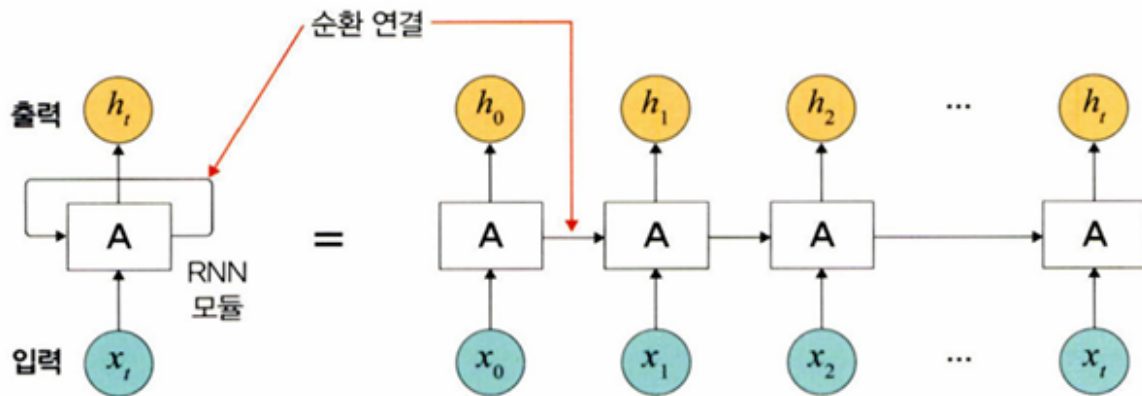
- 이미지에서 객체, 얼굴, 장면을 인식하기 위해 패턴을 찾는 데 유용
- ex) LeNet-5, AlexNet, VGG, GoogLeNet, ResNet
- 기존 신경망과의 차별성
 - 각 층의 입출력 형상 유지

- 이미지의 공간 정보를 유지하면서 인접 이미지와 차이가 있는 특징을 효과적으로 인식
- 복수 필터로 이미지의 특징 추출 및 학습
- 풀링층: 추출한 이미지의 특징을 모으고 강화
- 필터를 공유 파라미터로 사용 → 파라미터 적음

4.3.3. 순환 신경망(Recurrent Neural Network, RNN)

시계열 데이터(음악, 영상 등) 같은 시간 흐름에 따라 변화하는 데이터를 학습하기 위한 인공 신경망

- '순환'=자기 자신을 참조: 현재 결과가 이전 결과와 연관이 있음

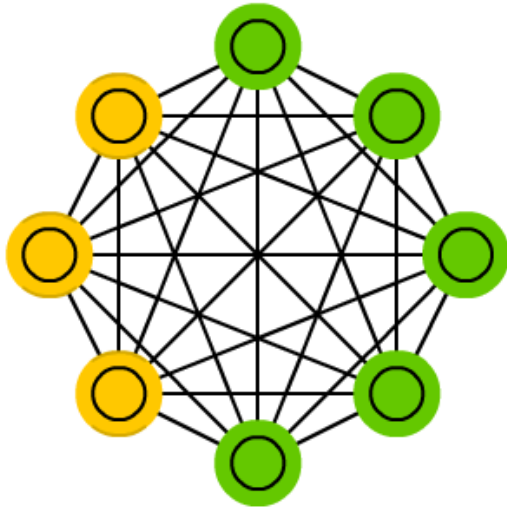


- 시간성(temporal property)을 가진 데이터가 많음
- 시간성 정보를 이용
- 동적 데이터, 길이가 가변적
- 매우 긴 데이터를 처리하는 연구가 활발히 진행 중
- 자연어 처리 분야와 궁합이 맞음
ex) 언어 모델링, 텍스트 생성, 자동 번역, 음성 인식, 이미지 캡션 생성 등

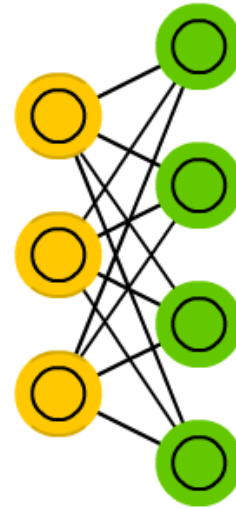
⚠ 기울기 소멸 문제로 학습이 제대로 되지 않는 문제 → 해결: LSTM(Long-Short Term Memory) (메모리 개념 도입)

4.3.4. 제한된 볼츠만 머신

Boltzmann Machine (BM)



Restricted BM (RBM)



볼츠만 머신(Boltzmann machine)

- 가시층(visible layer)과 은닉층으로 구성된 모델

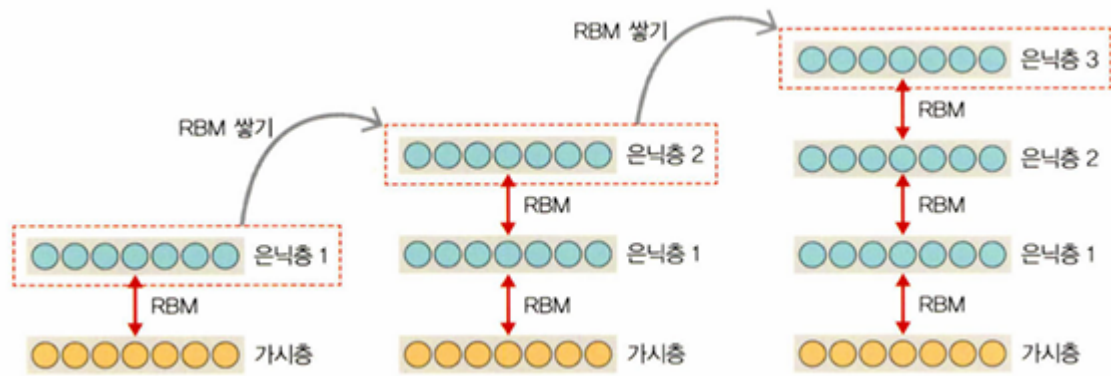
제한된 볼츠만 머신(Restricted BM, RBM)

- 가시층끼리, 은닉층끼리의 연결 없이 가시층+은닉층 연결만 있는 모델
- 사용: 차원 감소, 분류, 선형 회귀 분석, 협업 필터링, 특성 값 학습(feature learning), 주제 모델링(topic modelling)
- 기울기 소멸 문제 해결을 위해 사전 학습 용도로 활용 가능
- 심층 신뢰 신경망(DBN) 요소로 활용
- CNN, RNN에 비해 RBM, DBN은 상대적으로 많이 사용하진 않음

4.3.5. 심층 신뢰 신경망

사전 훈련된 제한된 볼츠만 머신(입력층+은닉층)을 층층이 쌓아 올린 구조

- 레이블이 없는 데이터에 대한 비지도 학습 가능
→ 부분적인 이미지에서 전체를 연상하는 일반화, 추상화 과정 구현 시 유용
- 학습 절차



이전 은닉층이 다음 은닉층의 가시층이 되는 것...으로 느껴짐

1. 가시층과 은닉층1에 제한된 볼츠만 머신 사전 훈련
2. 첫 번째 층 입력 데이터와 파라미터 고정, 두 번째 층 제한된 볼츠만 머신 사전 훈련
3. 원하는 층 개수만큼 제한된 볼츠만 머신을 쌓아 올려 전체 DBN 완성

- 특징

- 계층적 구조: 순차적으로 심층 신뢰 신경망을 학습시키면서 계층적 구조 생성
- 비지도 학습으로 학습
- 위로 올라갈수록 추상적 특성 추출
- 학습된 가중치를 다층 퍼셉트론의 가중치 초기값으로 사용